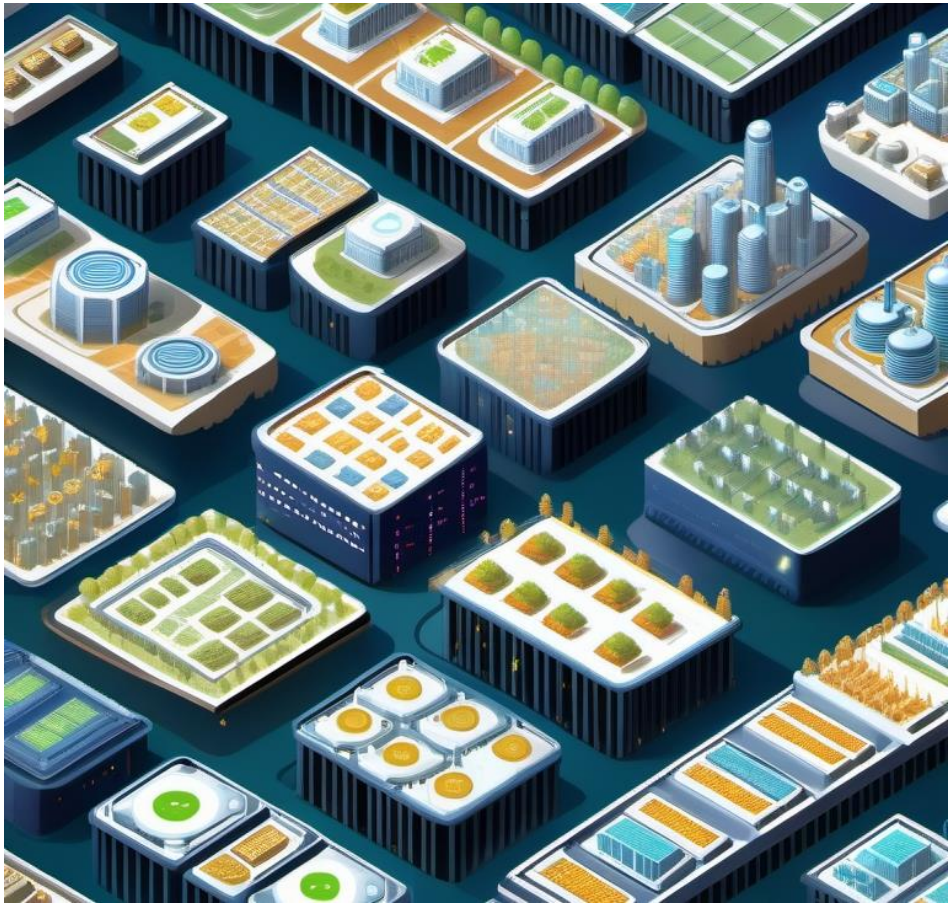


GESTIÓN DE SISTEMSA DE DATOS MASIVOS

INDEX



01. Introduction

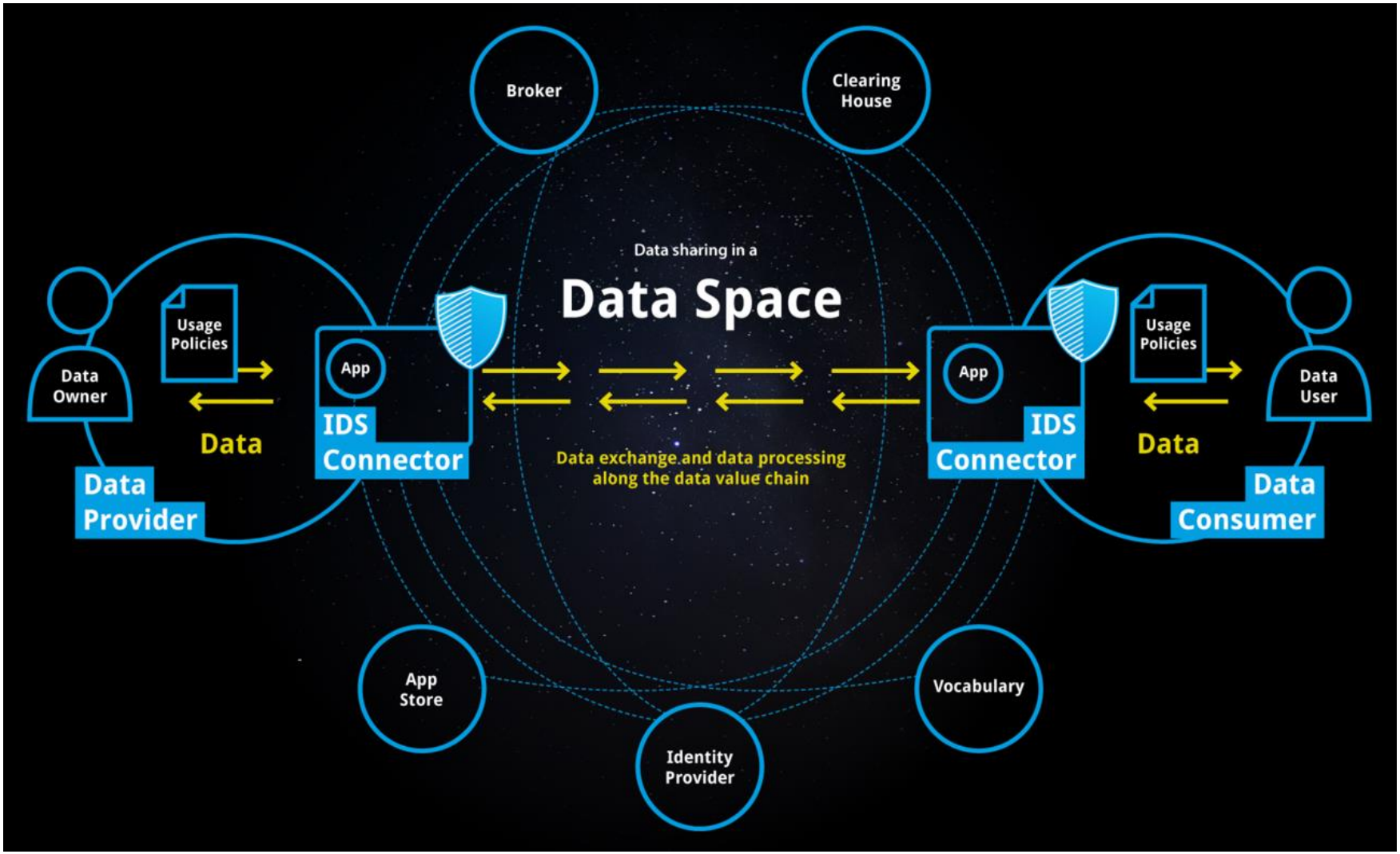
02. RDF generation

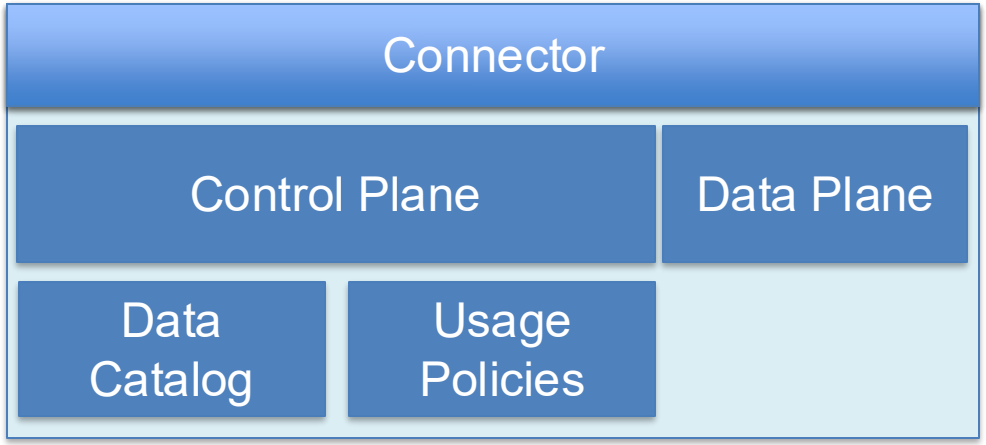
03. Triplestores

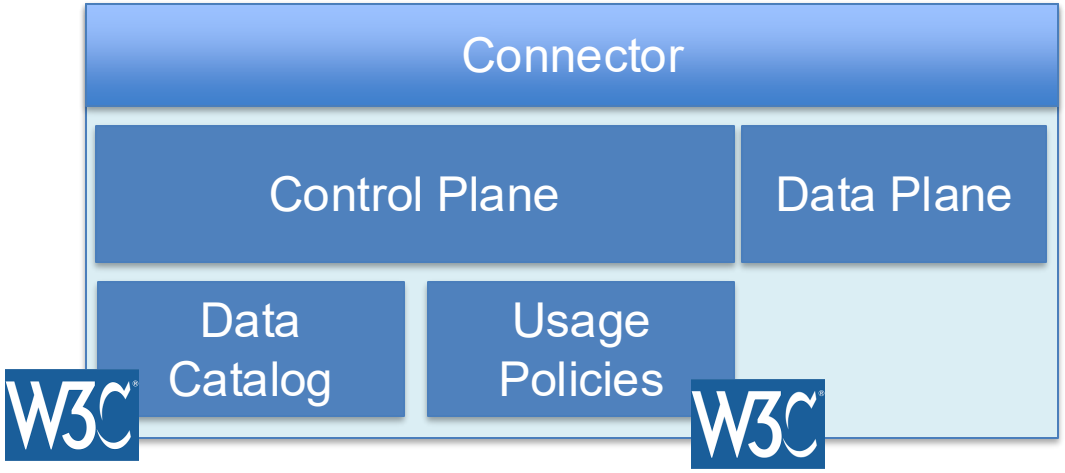
04. SPARQL

Summary

- RDF is a graph that is expressed with triples.
  - The URIs of the network can be prefixed.
- Subjects
  - They identify a real world entity
  - Must be **URIs or blank nodes**
- Predicate
  - Must be **URIs defined in a vocabulary/ontology**
  - Data properties: if the range of the predicate is a literal
  - Object properties: if the range of the predicate is a URI or blank node
  - rdf:type or a is the predicate used to assign a class to a subject
- Objects
  - Can be **URIs** (which can be subjects or finite classes in an ontology), **blank nodes** (which are themselves subjects), **or Literals** (string, string with type, or primitive type).


Broker
Clearing House
Data sharing in a
Data Space
Usage Policies
Data Owner
App
IDS Connector
Data
Data Provider
Data exchange and data processing along the data value chain
App
IDS Connector
Usage Policies
Data User
Data
Data Consumer
App Store
Identity Provider
Vocabulary


Connector
Control Plane
Data Plane
Data Catalog
Usage Policies



Interoperability at Discovery

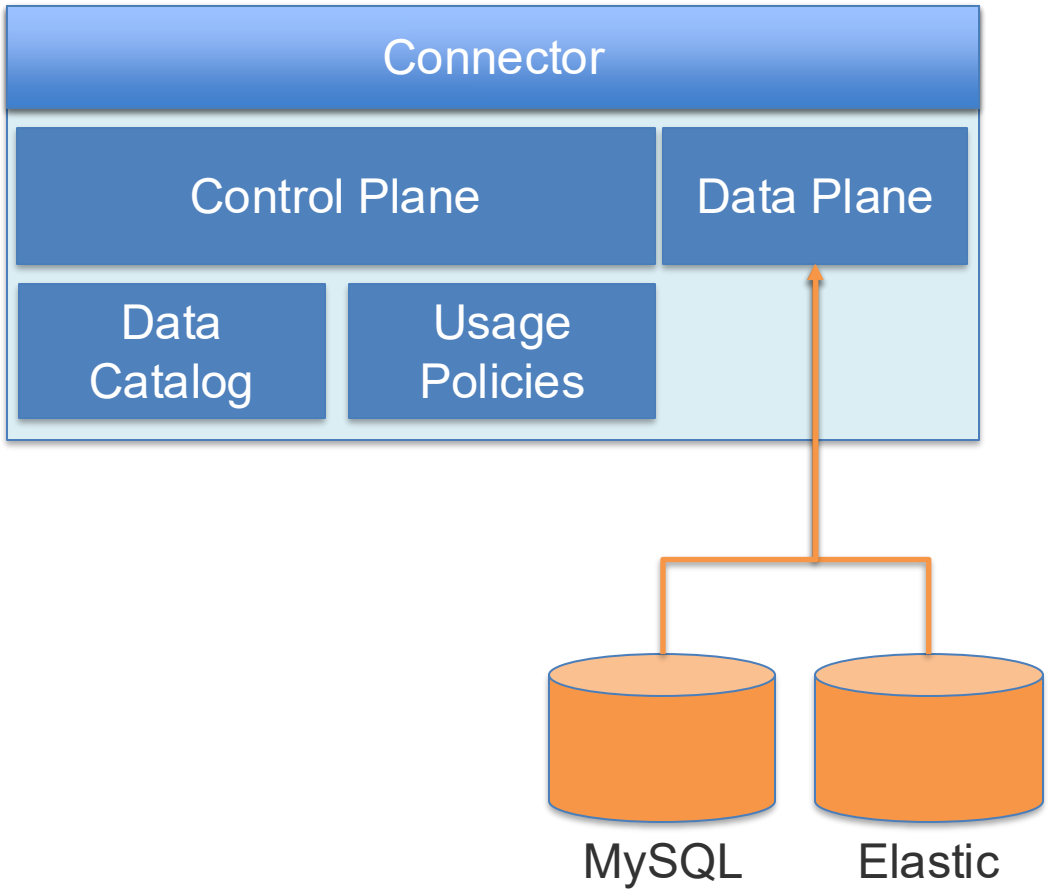
Interoperability at Contracting

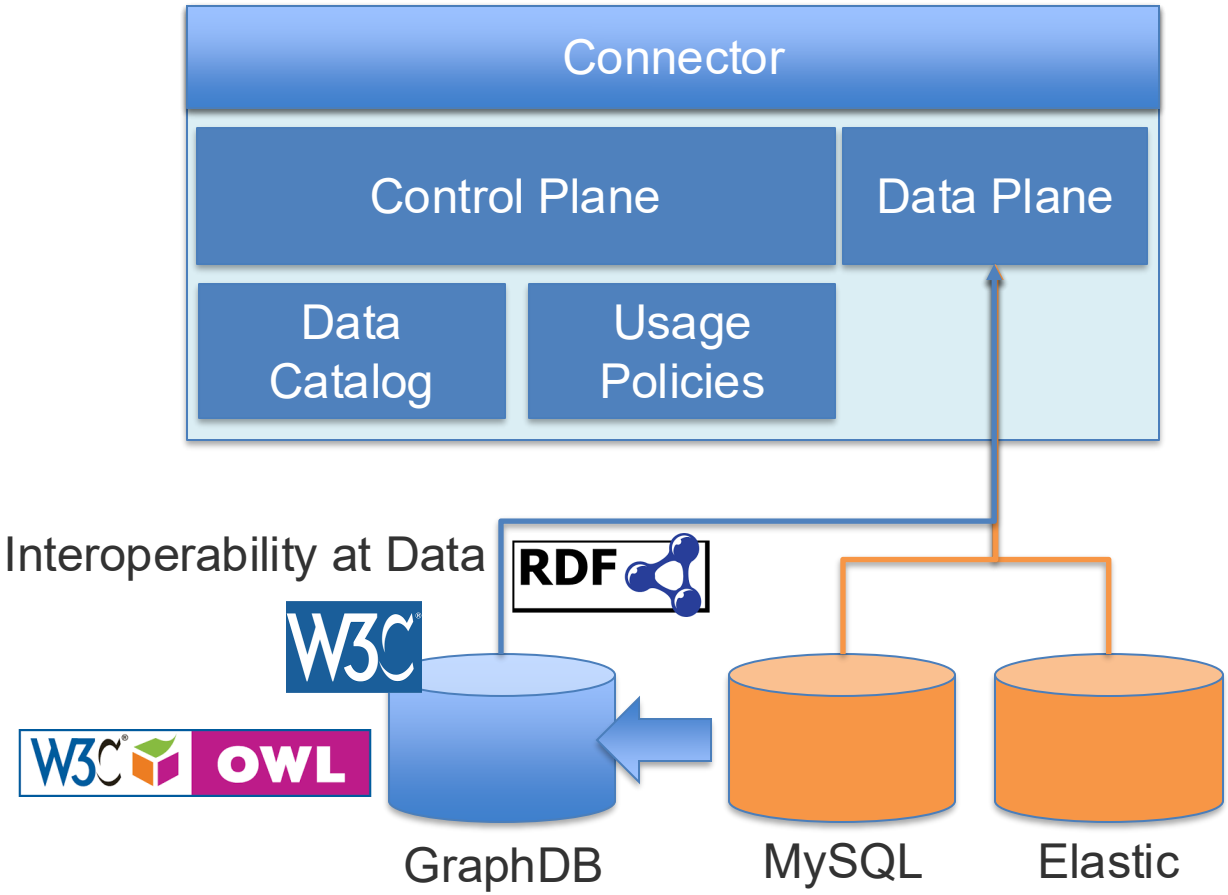
Data Catalog Vocabulary (DCAT) - Version 3

W3C Recommendation 22 August 2024

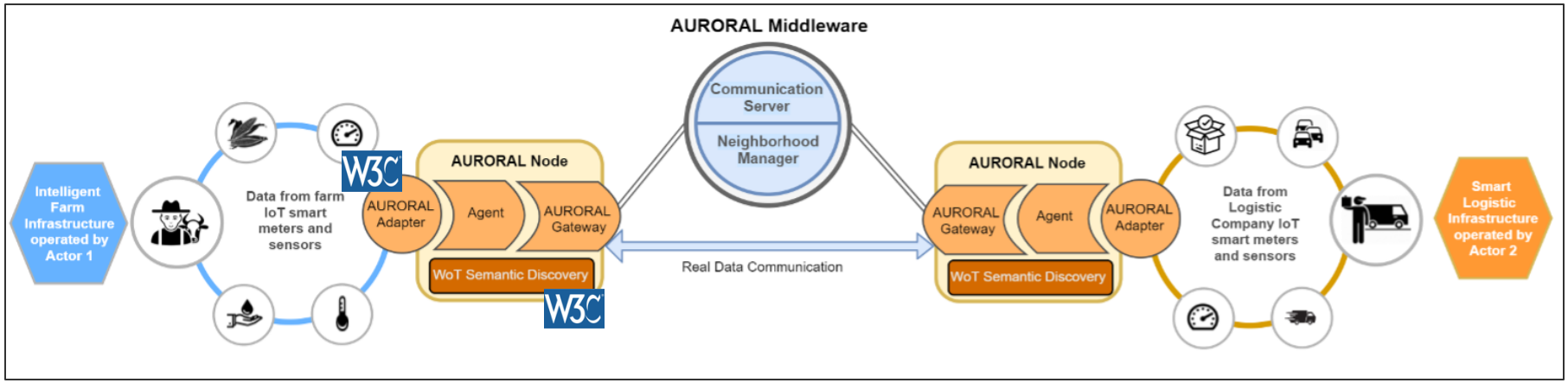
ODRL Information Model 2.2

W3C Recommendation 15 February 2018


Connector
Control Plane
Data Plane
Data Catalog
Usage Policies
MySQL
Elastic

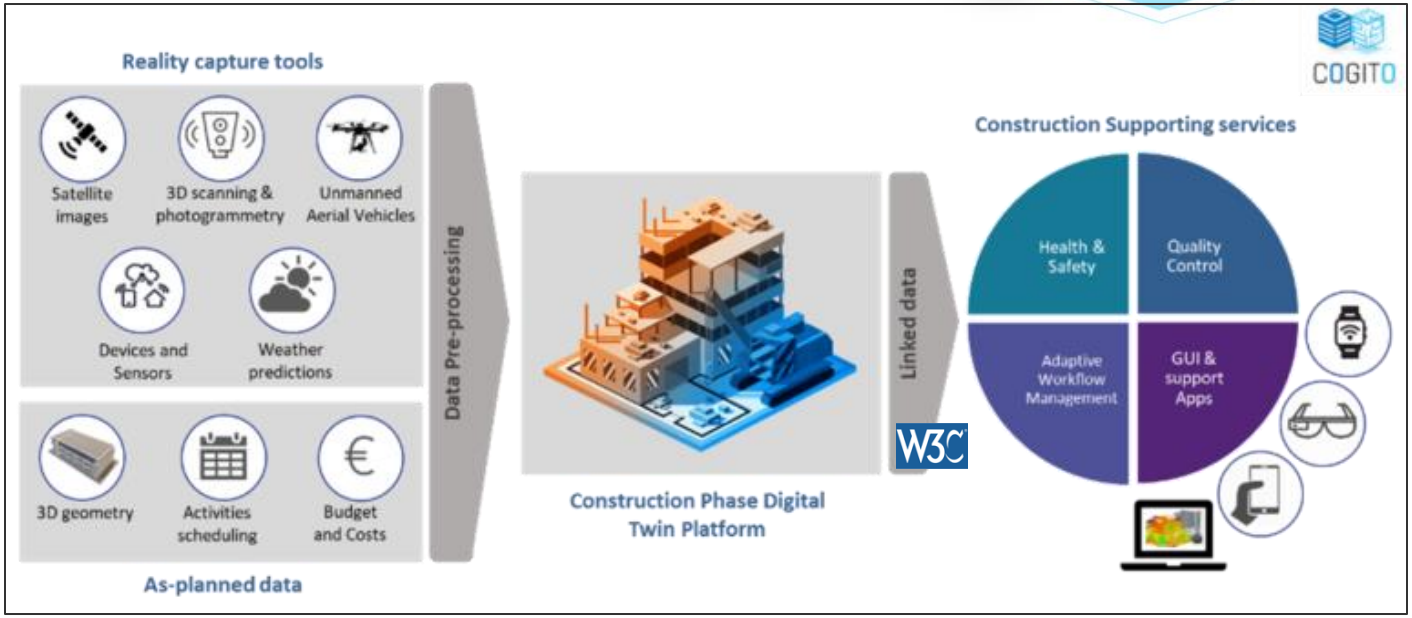

Connector
Control Plane
Data Plane
Data Catalog
Usage Policies
Interoperability at Data
RDF
W3C
W3C OWL
GraphDB
MySQL
Elastic

AURORAL



Cimmino, A., Cano-Benito, J., García-Castro, R. (2025). The AURORAL Privacy Approach for Smart Communities Based on ODRL. In: Presser, M., Skarmeta, A., Krco, S., González Vidal, A. (eds) Global Internet of Things and Edge Computing Summit. GIECS 2024. Communications in Computer and Information Science, vol 2328. Springer, Cham. https://doi.org/10.1007/978-3-031-78572-6_6

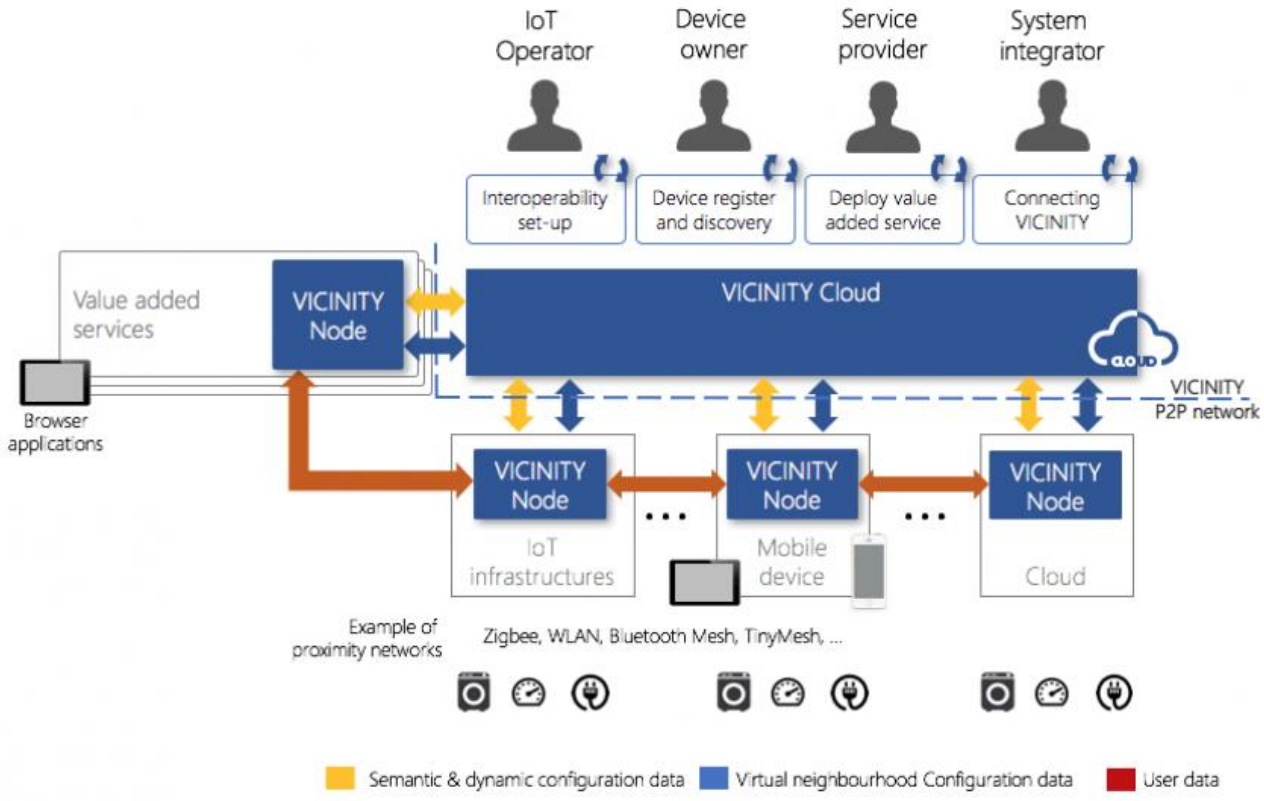
COGITO



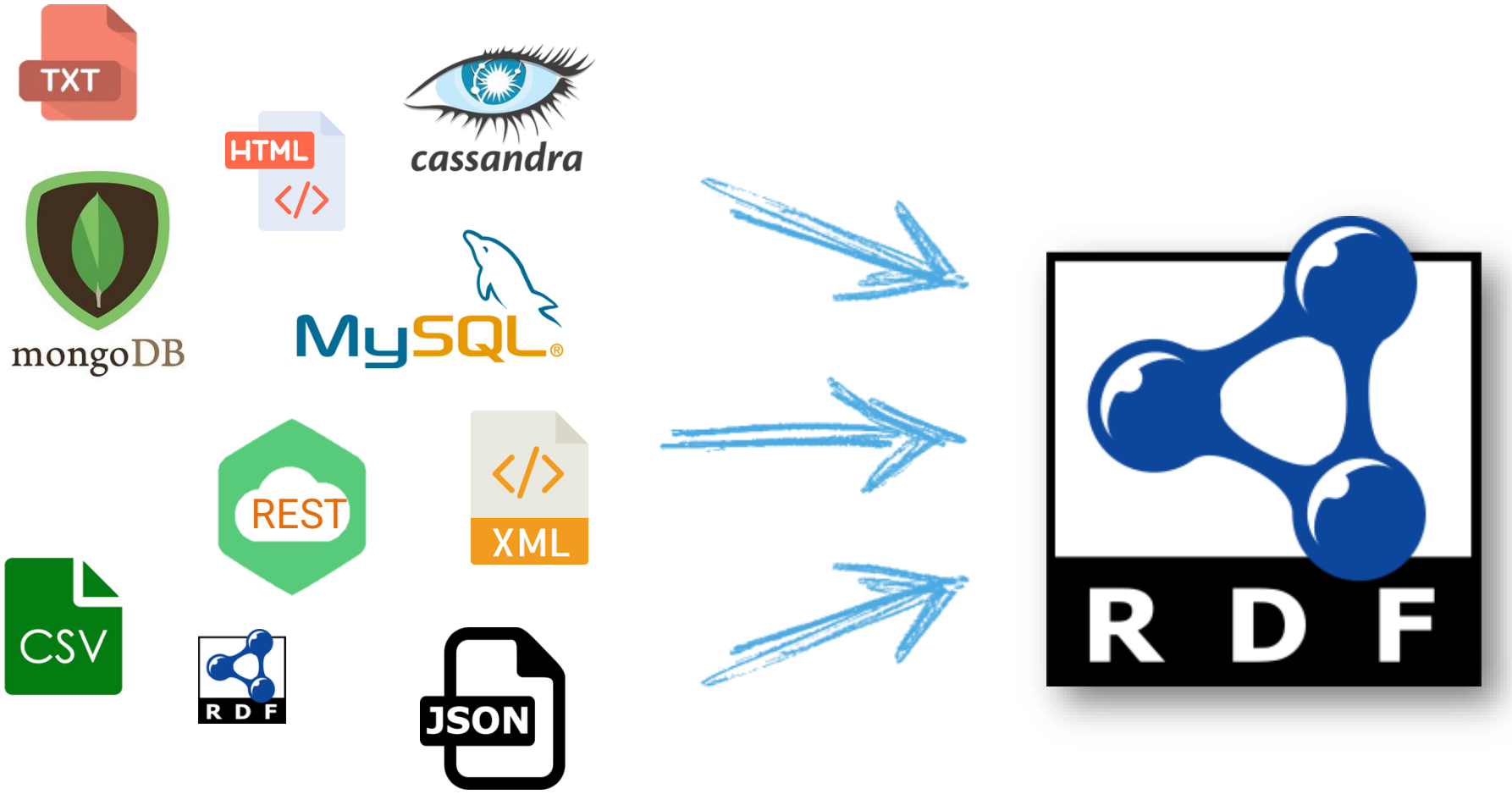
https://cogito-project.eu/welcome/

VICINITY 2020

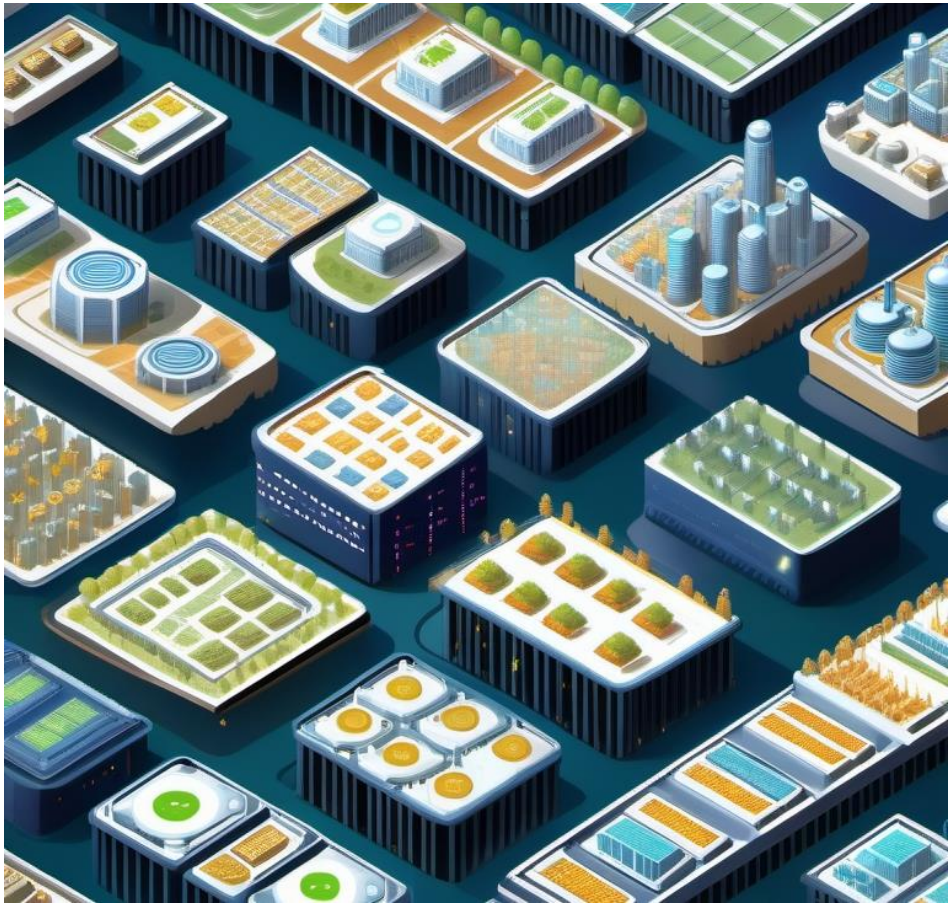
IoT European Platforms Initiative



RDF Generation



INDEX



01. Introduction

02. RDF generation

03. Triplestores

04. SPARQL

RDF & SPARQL

Well-know libraries



https://github.com/RDFLib/sparqlwrapper

https://rdflib.readthedocs.io/en/stable/



https://jena.apache.org/

RDF Generation



https://rml.io/

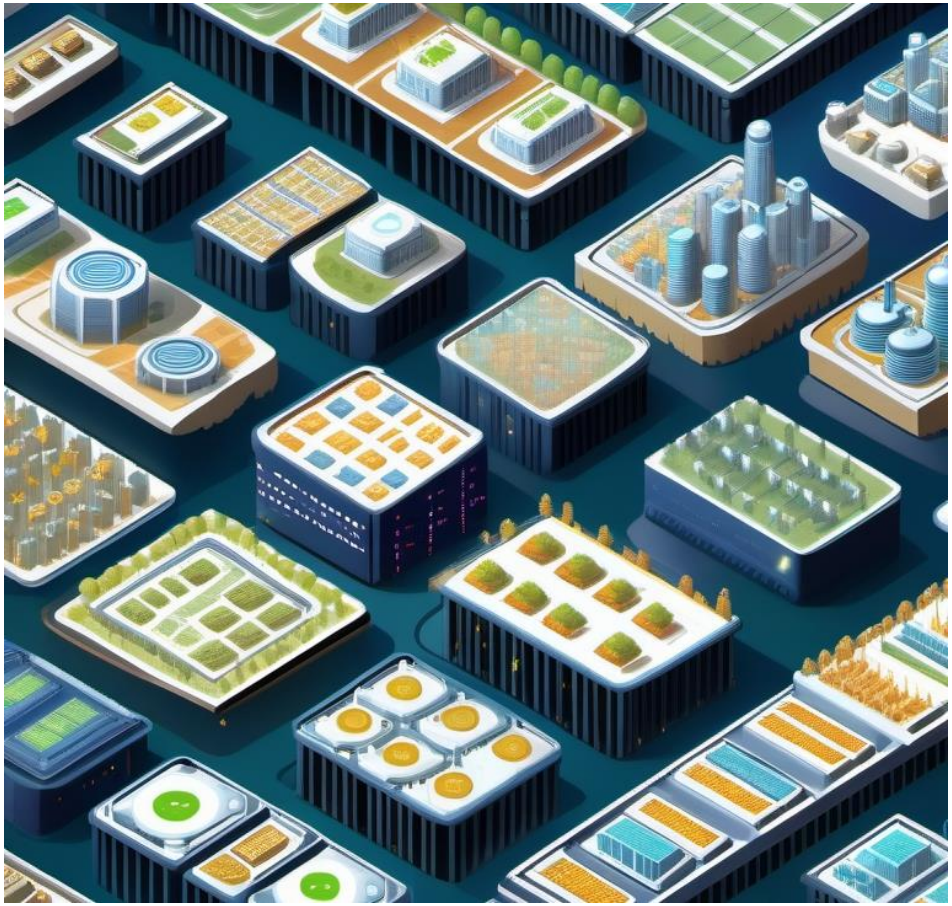


https://helio-playground.linkeddata.es/



https://openrefine.org/

INDEX



01. Introduction

02. RDF generation

03. Triplestores

04. SPARQL

Triplestores

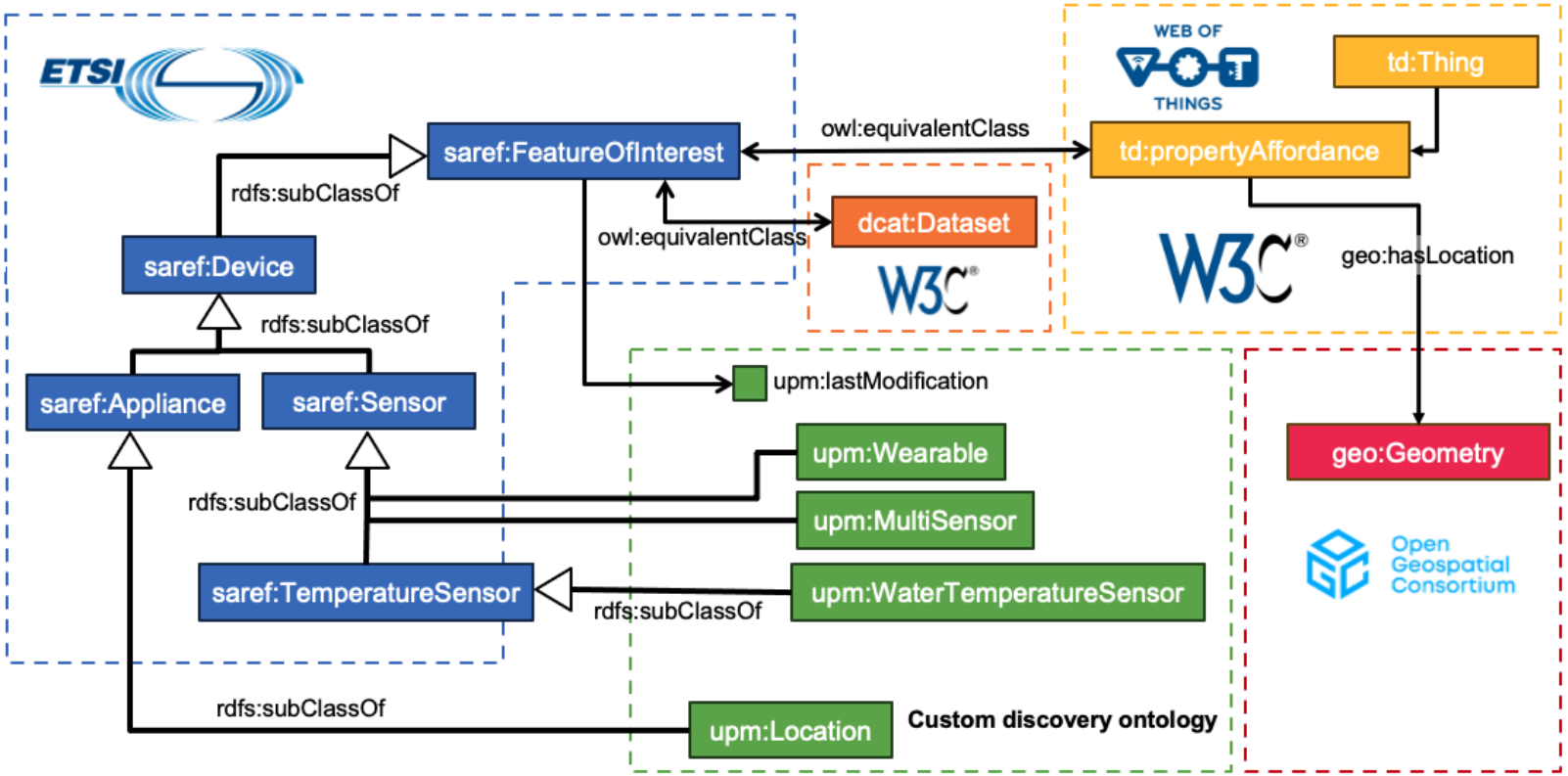
These are the databases for RDF



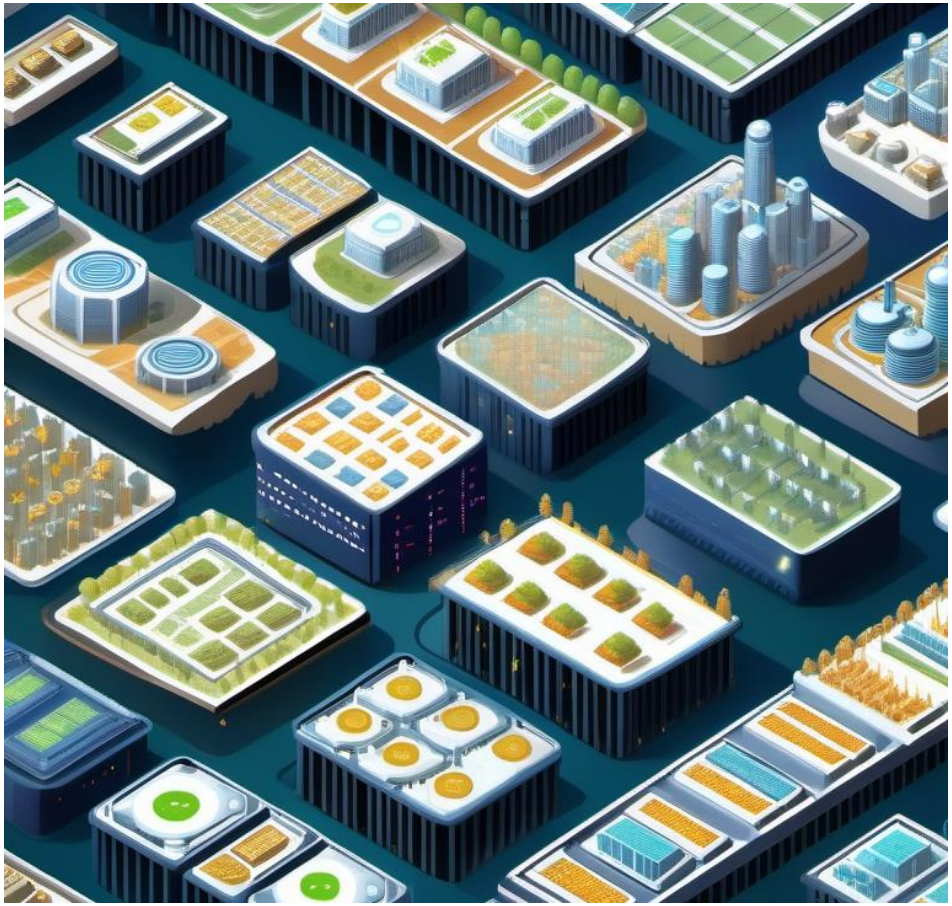
Common features:

- Load data
- Visualize/navigate in data
- SPARQL (federated)
- Data validation

Example


ETSI
saref:FeatureOfInterest
owl:equivalentClass
td:Thing
td:propertyAffordance
rdfs:subClassOf
owl:equivalentClass
dcat:Dataset
saref:Device
geo:hasLocation
rdfs:subClassOf
saref:Appliance
saref:Sensor
upm:lastModification
upm:Wearable
geo:Geometry
rdfs:subClassOf
upm:MultiSensor
saref:TemperatureSensor
upm:WaterTemperatureSensor
rdfs:subClassOf
rdfs:subClassOf
upm:Location
Custom discovery ontology

INDEX



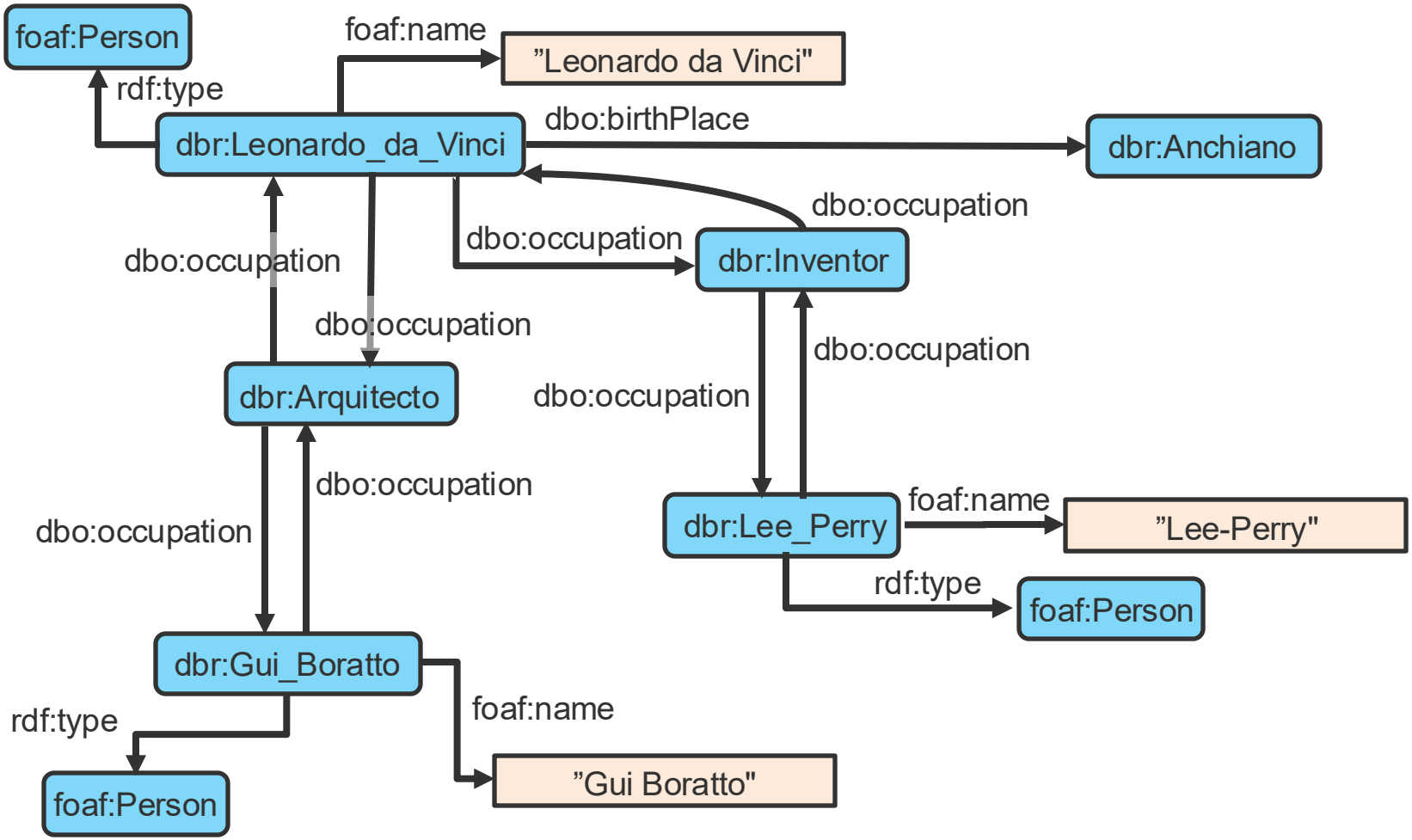
01. Introduction

02. RDF generation

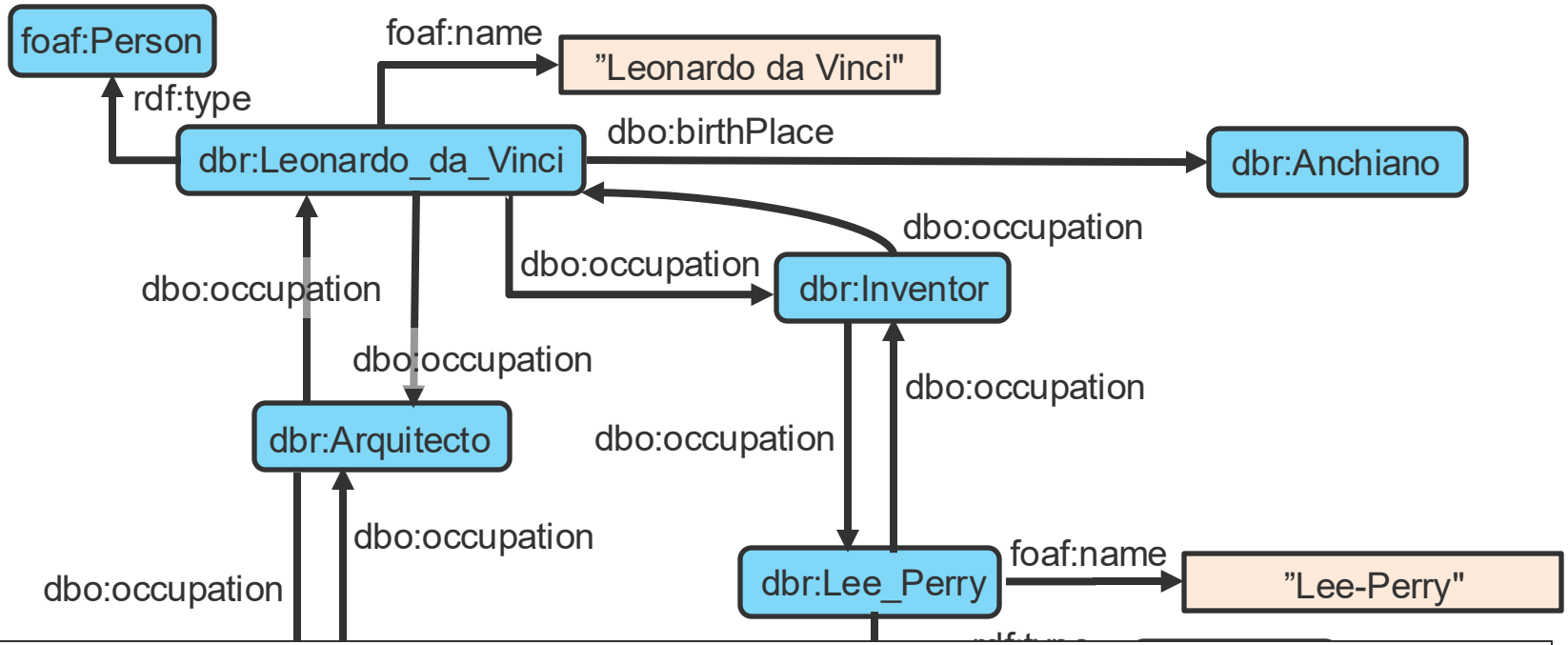
03. Triplestores

04. SPARQL

Data queries



Data queries

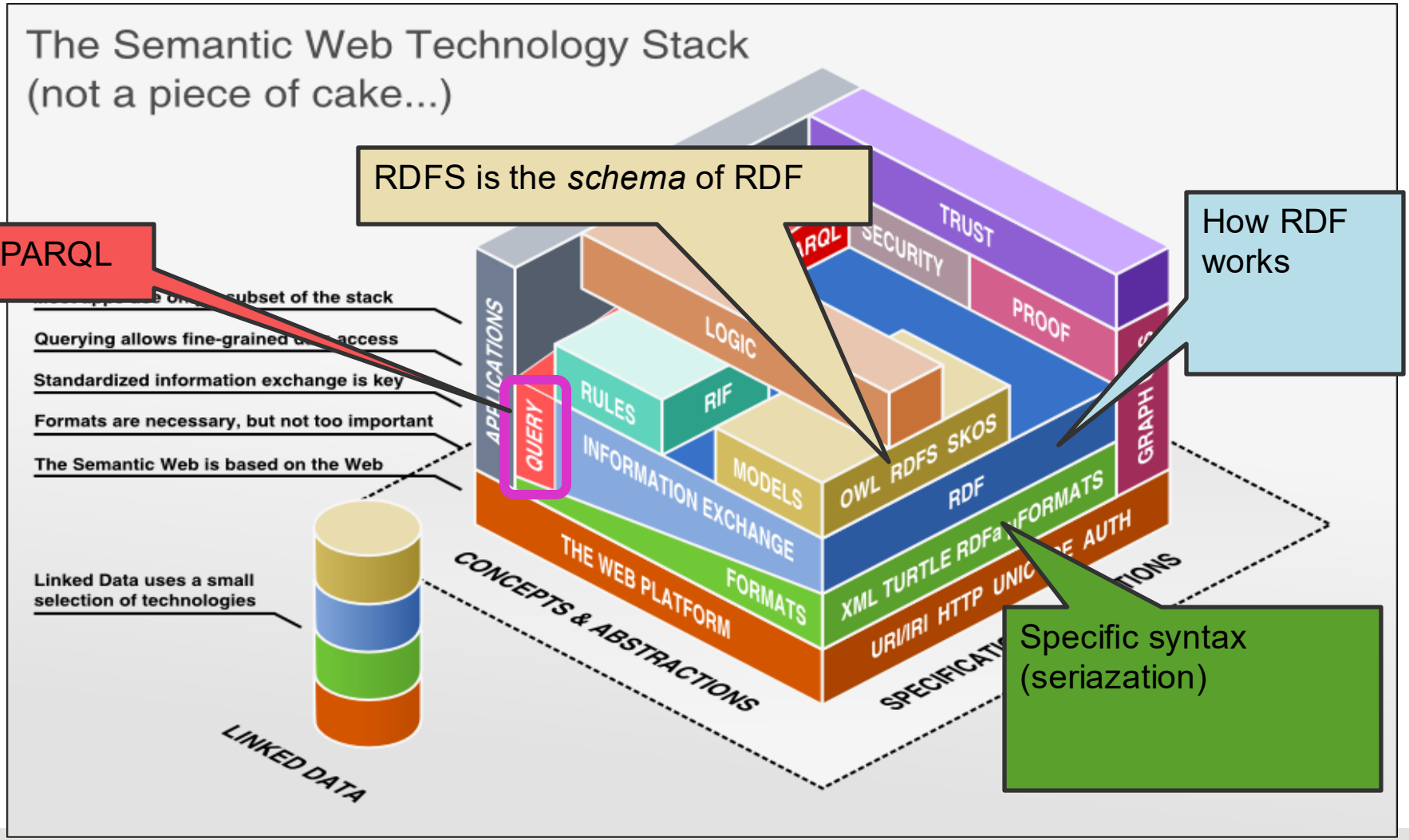




- What are people called?
- What are the names of people with a place of birth?
- Which people are architects and what are their names?
- How many people have more than one profession?

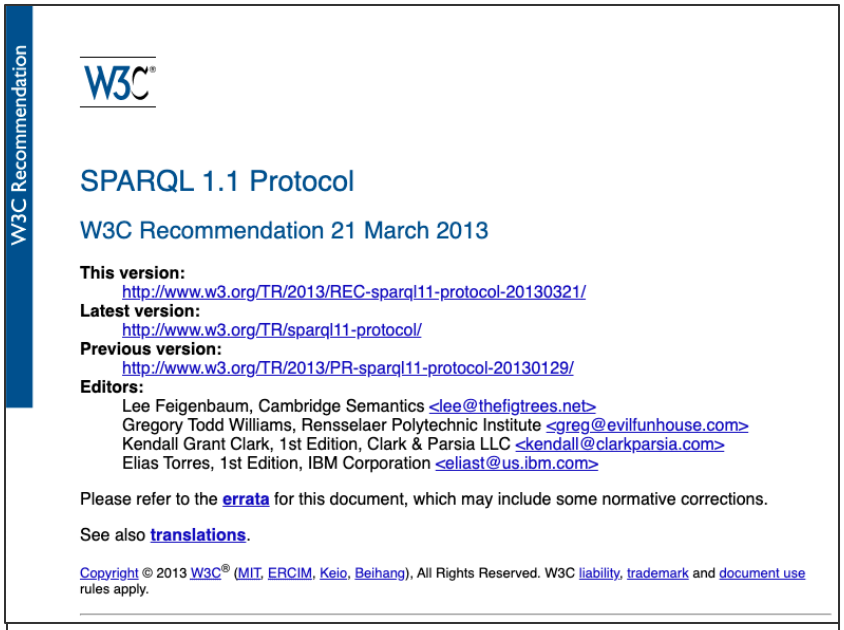
…

SPARQL


The Semantic Web Technology Stack
(not a piece of cake...)
RDFS is the *schema* of RDF
SPARQL
How RDF works
Specific syntax (seriazation)
subset of the stack
Querying allows fine-grained access
Standardized information exchange is key
Formats are necessary, but not too important
The Semantic Web is based on the Web
Linked Data uses a small selection of technologies
TRUST
SECURITY
PROOF
LOGIC
RULES
RIF
QUERY
INFORMATION EXCHANGE
MODELS
OWL RDFS SKOS
RDF
FORMATS
THE WEB PLATFORM
XML TURTLE RDFa
URI/IRI HTTP UNIC
AUTH
GRAPH
CONCEPTS & ABSTRACTIONS
LINKED DATA

SPARQL

Protocol & query language



W3C Recommendation

W3C

SPARQL 1.1 Protocol

W3C Recommendation 21 March 2013

This version:
http://www.w3.org/TR/2013/REC-sparql11-protocol-20130321/
Latest version:
http://www.w3.org/TR/sparql11-protocol/
Previous version:
http://www.w3.org/TR/2013/PR-sparql11-protocol-20130129/
Editors:
Lee Feigenbaum, Cambridge Semantics <lee@thefigtrees.net>
Gregory Todd Williams, Rensselaer Polytechnic Institute <greg@evilfunhouse.com>
Kendall Grant Clark, 1st Edition, Clark & Parsia LLC <kendall@clarkparsia.com>
Elias Torres, 1st Edition, IBM Corporation <eliast@us.ibm.com>

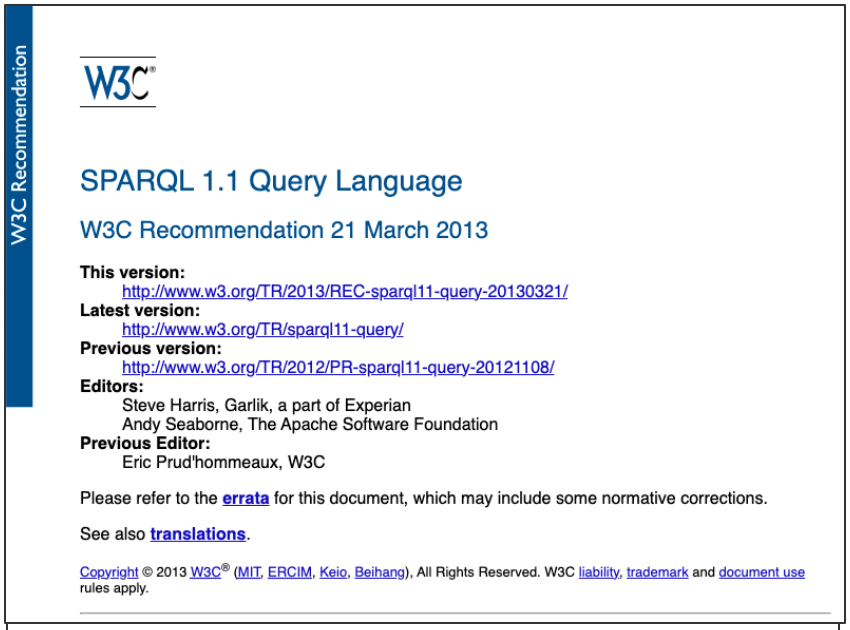
Please refer to the **errata** for this document, which may include some normative corrections.

See also **translations**.

Copyright © 2013 W3C® (MIT, ERCIM, Keio, Beihang), All Rights Reserved. W3C liability, trademark and document use rules apply.

https://www.w3.org/TR/sparql11-protocol/

- Send queries
- Results formats
- Query federation
- …



W3C Recommendation

W3C

SPARQL 1.1 Query Language

W3C Recommendation 21 March 2013

This version:
http://www.w3.org/TR/2013/REC-sparql11-query-20130321/
Latest version:
http://www.w3.org/TR/sparql11-query/
Previous version:
http://www.w3.org/TR/2012/PR-sparql11-query-20121108/
Editors:
Steve Harris, Garlik, a part of Experian
Andy Seaborne, The Apache Software Foundation
Previous Editor:
Eric Prud'hommeaux, W3C

Please refer to the **errata** for this document, which may include some normative corrections.

See also **translations**.

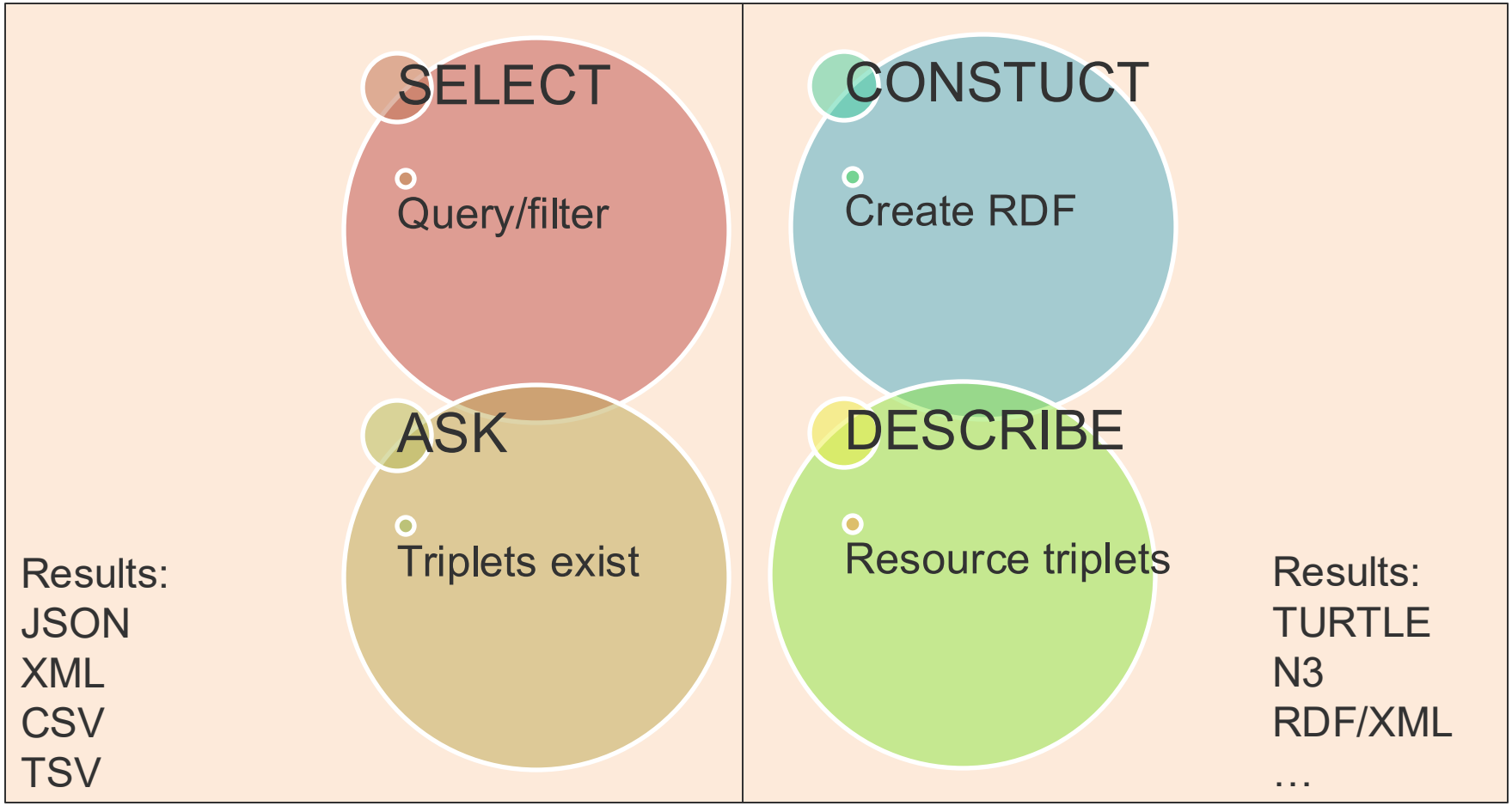
Copyright © 2013 W3C® (MIT, ERCIM, Keio, Beihang), All Rights Reserved. W3C liability, trademark and document use rules apply.

https://www.w3.org/TR/sparql11-query/

- Different query types
- Syntax of queries
- Functions
- …

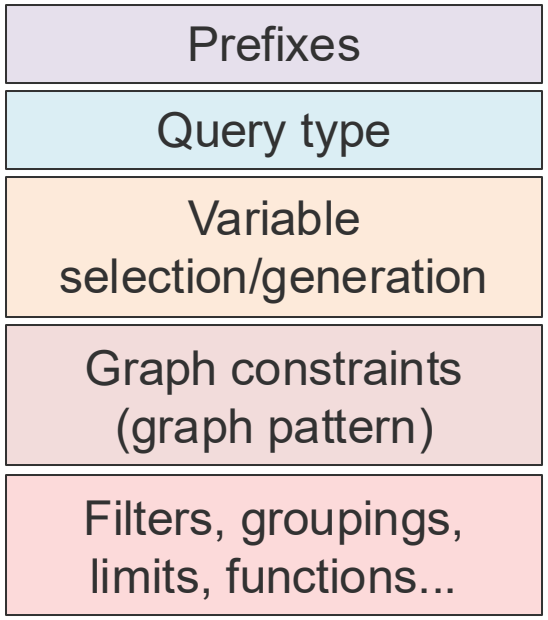
SPARQL

Query types



SPARQL

General syntax

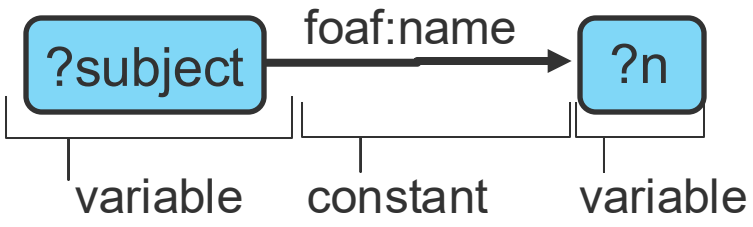


SPARQL

Triple patters and Graph patterns

- Triple pattern: is a pattern in the form of a triplet
  - Encodes a constraint on any combination of subject, predicate, or object
  - Variables are declared with a '?' in front of a set of characters, e.g., ?a, ?subject, ...
  - Constants can be URIs or literals.

- Example: any subject that has a name



Solving a triple pattern (I)



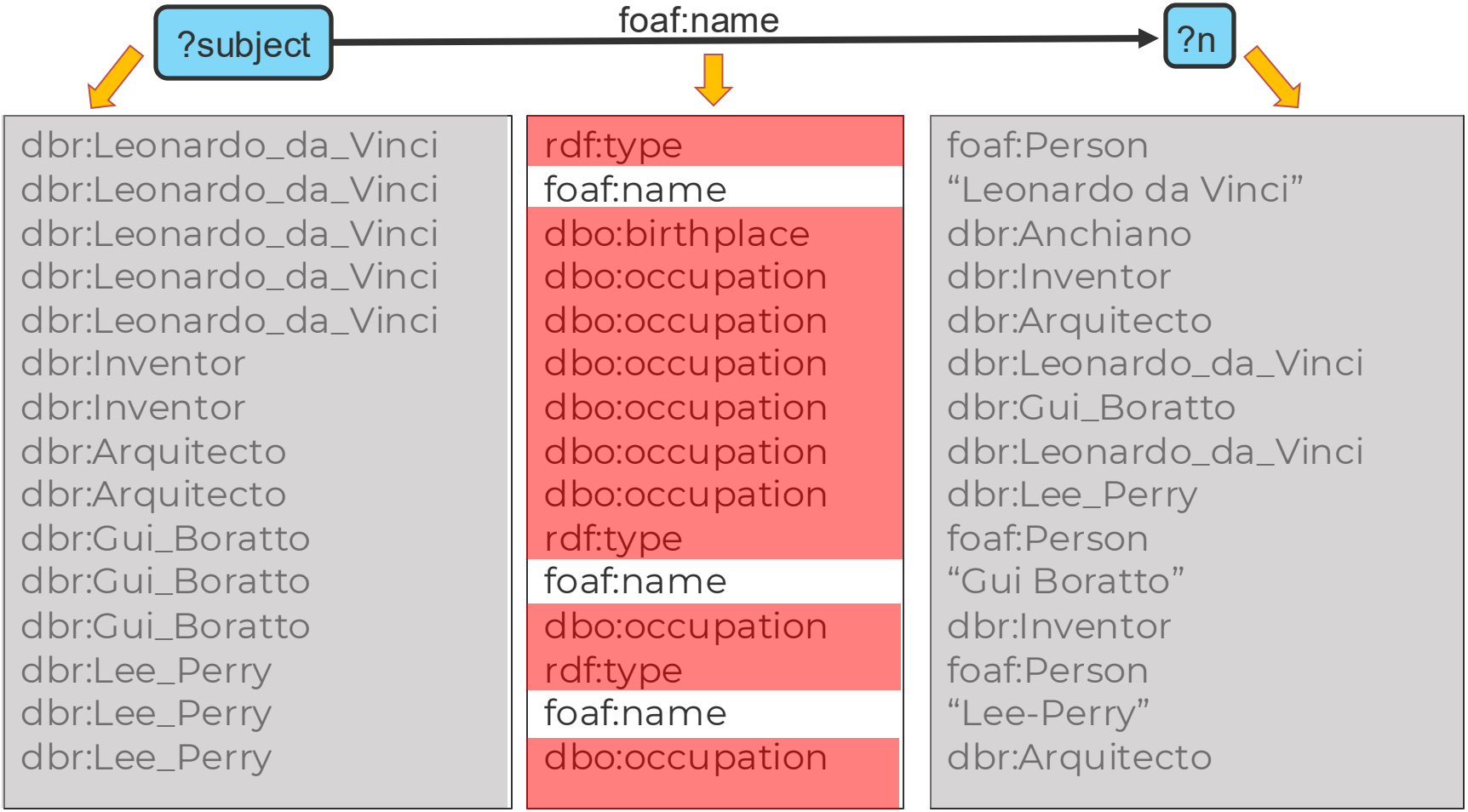
dbr:Leonardo_da_Vinci	rdf:type	foaf:Person
dbr:Leonardo_da_Vinci	foaf:name	“Leonardo da Vinci”
dbr:Leonardo_da_Vinci	dbo:birthplace	dbr:Anchiano
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Inventor
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Arquitecto
dbr:Inventor	dbo:occupation	dbr:Leonardo_da_Vinci
dbr:Inventor	dbo:occupation	dbr:Gui_Boratto
dbr:Arquitecto	dbo:occupation	dbr:Leonardo_da_Vinci
dbr:Arquitecto	dbo:occupation	dbr:Lee_Perry
dbr:Gui_Boratto	rdf:type	foaf:Person
dbr:Gui_Boratto	foaf:name	“Gui Boratto”
dbr:Gui_Boratto	dbo:occupation	dbr:Inventor
dbr:Lee_Perry	rdf:type	foaf:Person
dbr:Lee_Perry	foaf:name	“Lee-Perry”
dbr:Lee_Perry	dbo:occupation	dbr:Arquitecto

Solving a triple pattern (I)



dbr:Leonardo_da_Vinci	rdf:type	foaf:Person
dbr:Leonardo_da_Vinci	foaf:name	“Leonardo da Vinci”
dbr:Leonardo_da_Vinci	dbo:birthplace	dbr:Anchiano
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Inventor
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Arquitecto
dbr:Inventor	dbo:occupation	dbr:Leonardo_da_Vinci
dbr:Inventor	dbo:occupation	dbr:Gui_Boratto
dbr:Arquitecto	dbo:occupation	dbr:Leonardo_da_Vinci
dbr:Arquitecto	dbo:occupation	dbr:Lee_Perry
dbr:Gui_Boratto	rdf:type	foaf:Person
dbr:Gui_Boratto	foaf:name	“Gui Boratto”
dbr:Gui_Boratto	dbo:occupation	dbr:Inventor
dbr:Lee_Perry	rdf:type	foaf:Person
dbr:Lee_Perry	foaf:name	“Lee-Perry”
dbr:Lee_Perry	dbo:occupation	dbr:Arquitecto

Solving a triple pattern (I)

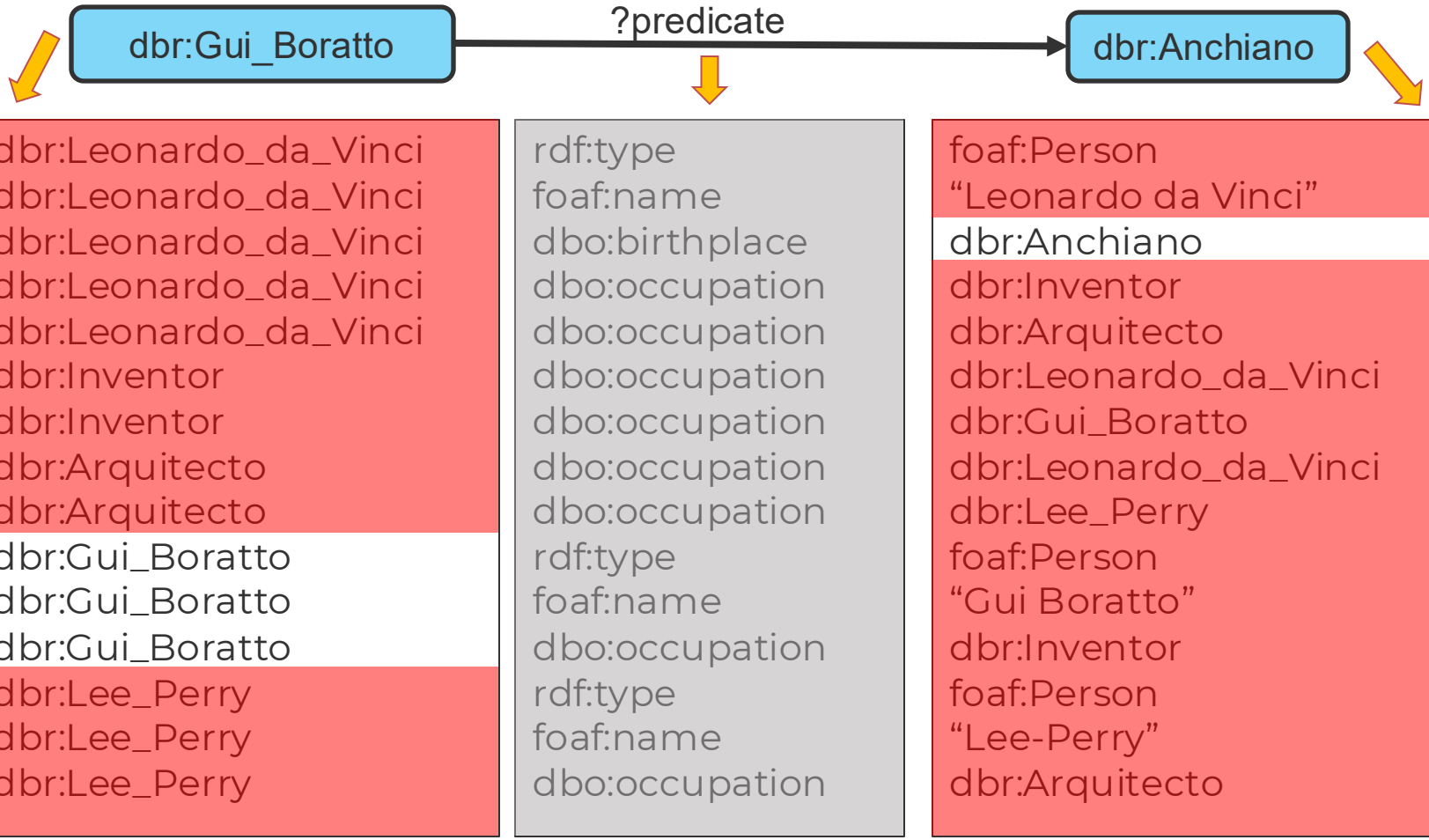


Solving a triple pattern (I)

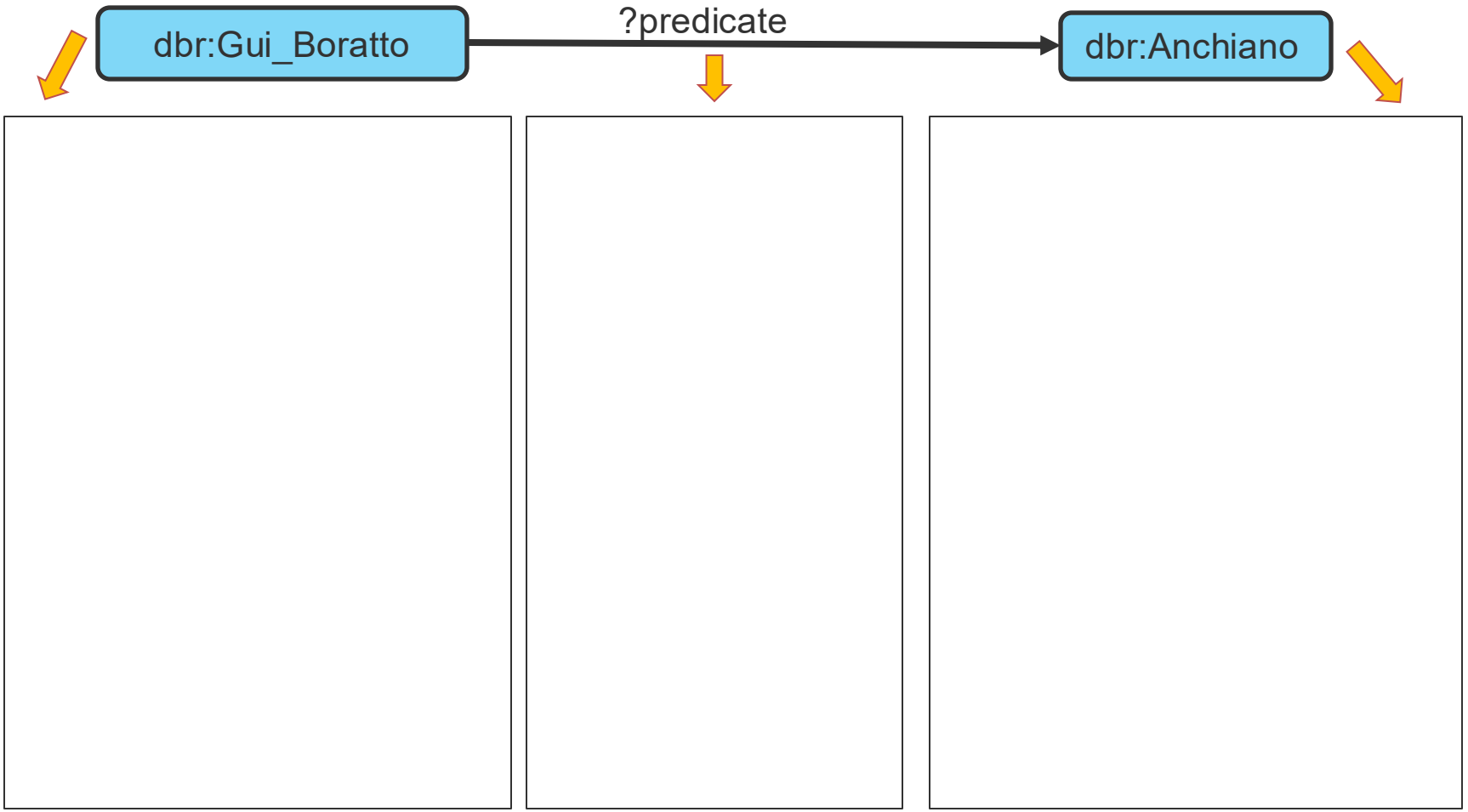


dbr:Leonardo_da_Vinci	foaf:name	“Leonardo da Vinci”
dbr:Gui_Boratto	foaf:name	“Gui Boratto”
dbr:Lee_Perry	foaf:name	“Lee-Perry”

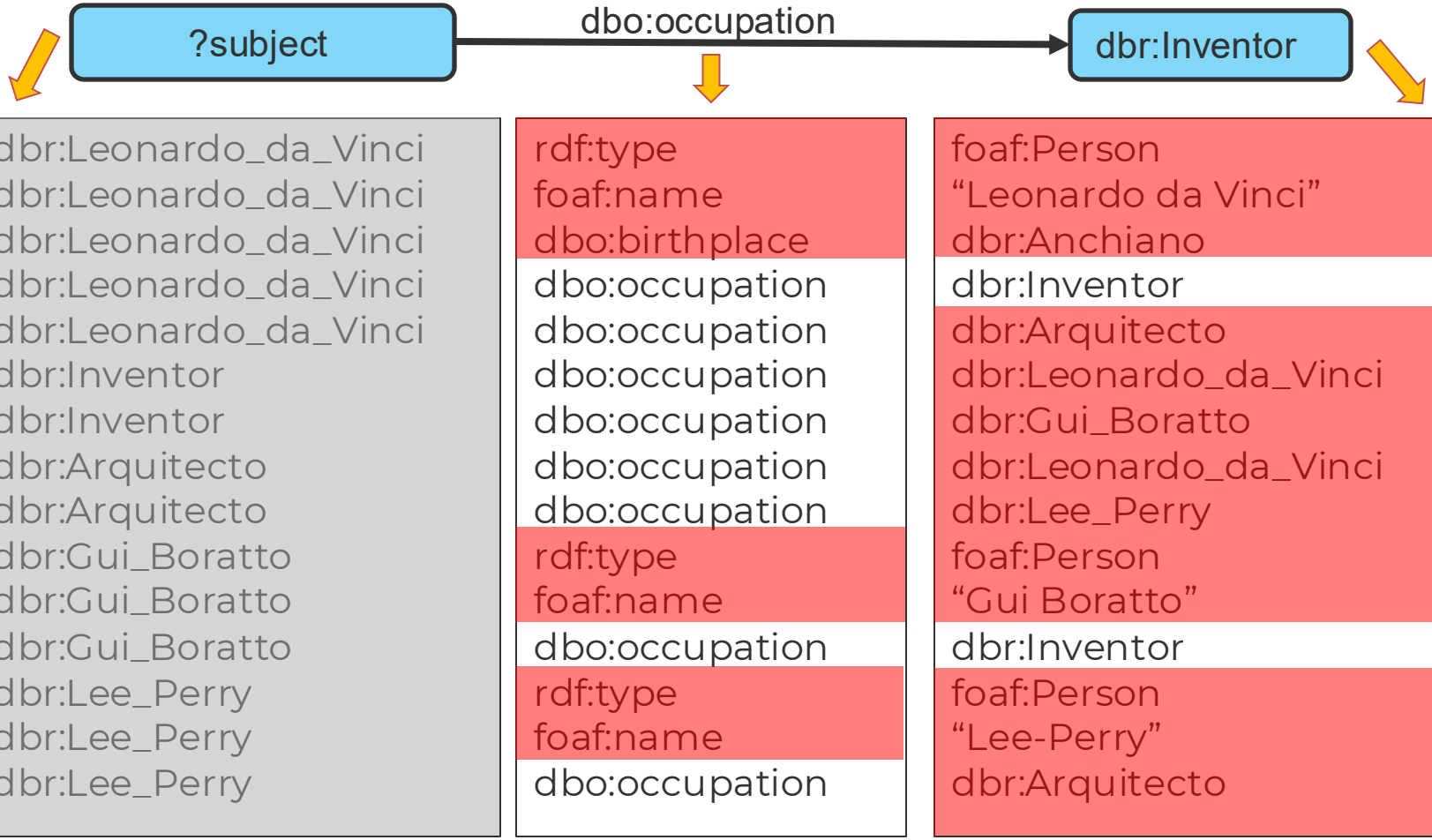
Solving a triple pattern (II)



Solving a triple pattern (II)



Solving a triple pattern (III)



Solving a triple pattern (III)



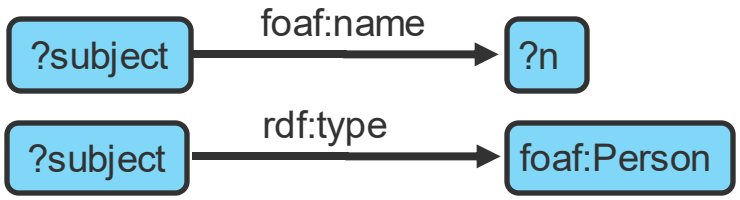
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Inventor
dbr:Gui_Boratto	dbo:occupation	dbr:Inventor

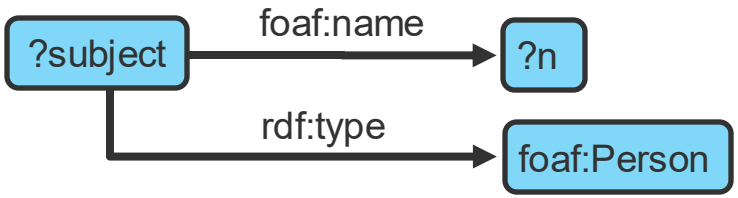
SPARQL

Triple patters y Graph patterns

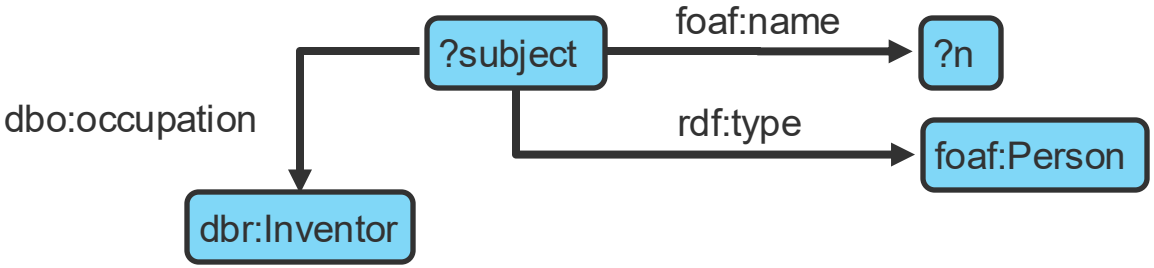
- Graph pattern: is a set of triple patterns
  - encodes a set of constraints on a graph

- Example: any subject that has a name and is a person.

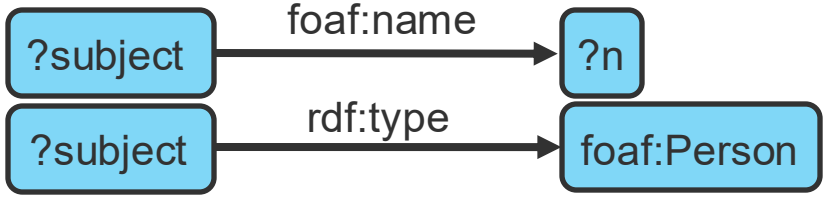


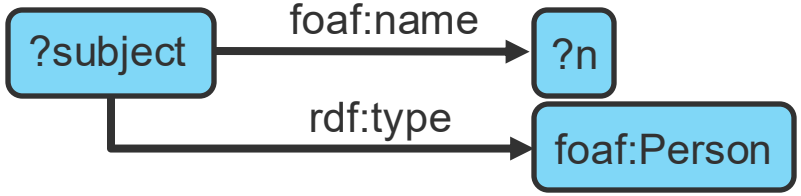


- Example: any subject that has a name and is a person and are inventors.



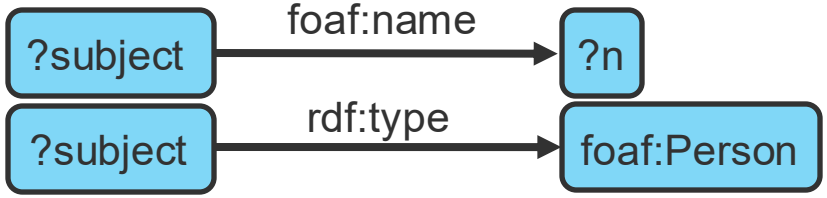
Solving a graph pattern (I)

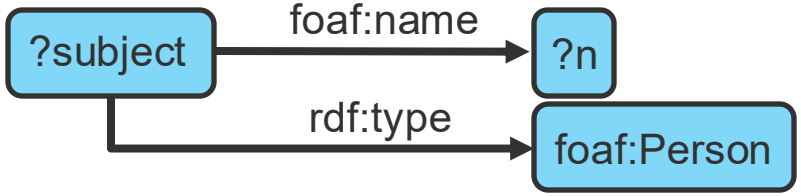




dbr:Leonardo_da_Vinci	rdf:type	foaf:Person
dbr:Leonardo_da_Vinci	foaf:name	“Leonardo da Vinci”
dbr:Leonardo_da_Vinci	dbo:birthplace	dbr:Anchiano
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Inventor
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Arquitecto
dbr:Inventor	dbo:occupation	dbr:Leonardo_da_Vinci
dbr:Inventor	dbo:occupation	dbr:Gui_Boratto
dbr:Arquitecto	dbo:occupation	dbr:Leonardo_da_Vinci
dbr:Arquitecto	dbo:occupation	dbr:Lee_Perry
dbr:Gui_Boratto	rdf:type	foaf:Person
dbr:Gui_Boratto	foaf:name	“Gui Boratto”
dbr:Gui_Boratto	dbo:occupation	dbr:Inventor
dbr:Lee_Perry	rdf:type	foaf:Person
dbr:Lee_Perry	foaf:name	“Lee-Perry”
dbr:Lee_Perry	dbo:occupation	dbr:Arquitecto

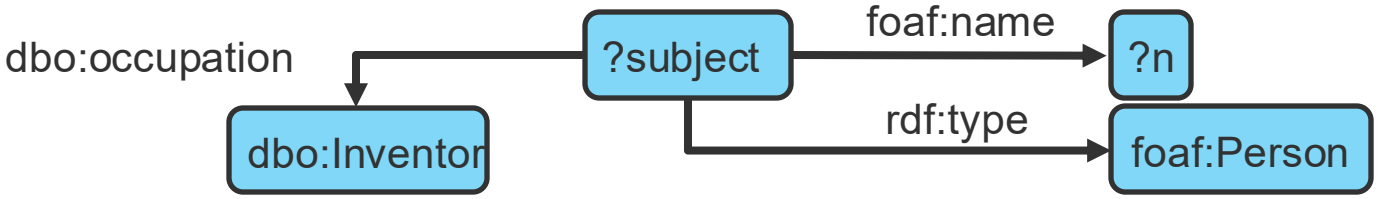
Solving a graph pattern (I)





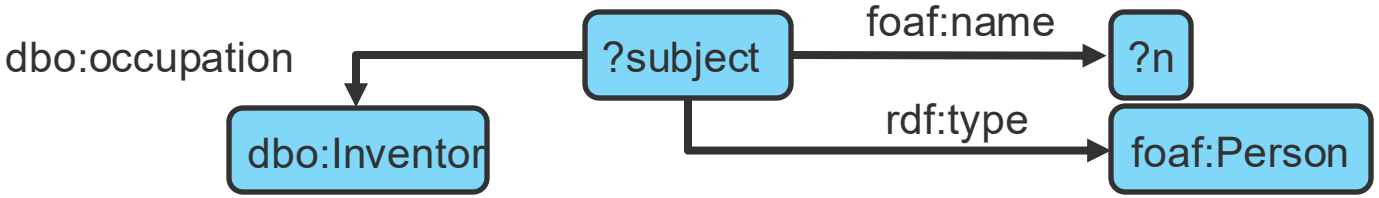
dbr:Leonardo_da_Vinci dbr:Leonardo_da_Vinci	rdf:type foaf:name	foaf:Person “Leonardo da Vinci”
dbr:Gui_Boratto dbr:Gui_Boratto	rdf:type foaf:name	foaf:Person “Gui Boratto”
dbr:Lee_Perry dbr:Lee_Perry	rdf:type foaf:name	foaf:Person “Lee-Perry”

Solving a graph pattern (II)



dbr:Leonardo_da_Vinci	rdf:type	foaf:Person
dbr:Leonardo_da_Vinci	foaf:name	“Leonardo da Vinci”
dbr:Leonardo_da_Vinci	dbo:birthplace	dbr:Anchiano
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Inventor
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Arquitecto
dbr:Inventor	dbo:occupation	dbr:Leonardo_da_Vinci
dbr:Inventor	dbo:occupation	dbr:Gui_Boratto
dbr:Arquitecto	dbo:occupation	dbr:Leonardo_da_Vinci
dbr:Arquitecto	dbo:occupation	dbr:Lee_Perry
dbr:Gui_Boratto	rdf:type	foaf:Person
dbr:Gui_Boratto	foaf:name	“Gui Boratto”
dbr:Gui_Boratto	dbo:occupation	dbr:Inventor
dbr:Lee_Perry	rdf:type	foaf:Person
dbr:Lee_Perry	foaf:name	“Lee-Perry”
dbr:Lee_Perry	dbo:occupation	dbr:Arquitecto

Solving a graph pattern (II)



dbr:Leonardo_da_Vinci	rdf:type	foaf:Person
dbr:Leonardo_da_Vinci	foaf:name	“Leonardo da Vinci”
dbr:Leonardo_da_Vinci	dbo:occupation	dbr:Inventor
dbr:Gui_Boratto	rdf:type	foaf:Person
dbr:Gui_Boratto	foaf:name	“Gui Boratto”
dbr:Gui_Boratto	dbo:occupation	dbr:Inventor

SPARQL

ASK, SELECT, DESCRIBE, CONSTRUCT

```
[PREFIXES]

ASK  {
      [Graph Patters]
}
```

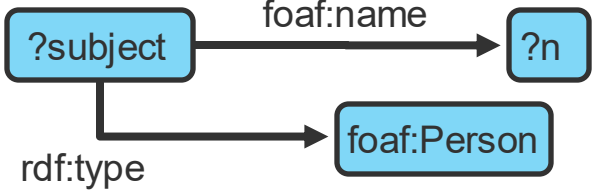
- Are there named subjects (foaf:name) ?




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

ASK {
      ?subject foaf:name ?name .
}
```

- Are there subjects that have a name (foaf:name) and are persons?




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

ASK {
      ?subject rdf:type foaf:Person .
      ?subject foaf:name ?name .
}
```

SPARQL

ASK, SELECT, DESCRIBE, CONSTRUCT

JSON

```
{ "head" : { } , "boolean" : true/false }
```

CSV

```
true/false
```

XML

```
<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
    <head></head>
    <boolean>true/false</boolean>
</sparql>
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

```
[PREFIXES]

SELECT DISTINCT [VARIABLES/BINDINGS] WHERE {
        [Graph Patters]
        [FILTER/BIND]
} [GROUP BY/HAVING/ORDER BY/LIMIT/OFFSET]
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Retrieve named subjects (foaf:name)




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?subject WHERE {
      ?subject foaf:name ?name .
}
```

- Finds all subjects that have a name (foaf:name) and returns both




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?subject ?name WHERE {
      ?subject foaf:name ?name .
}
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Retrieve different named subjects (foaf:name)




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject WHERE {
      ?subject foaf:name ?name .
}
```

- Finds all different subjects that have a name (foaf:name) and returns both




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject ?name
WHERE {
      ?subject foaf:name ?name .
}
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Finds all named subjects (foaf:name) and return only 10




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject WHERE {
      ?subject foaf:name ?name .
} LIMIT 10
```

- Finds all named subjects (foaf:name) and return only 10 skipping first 5



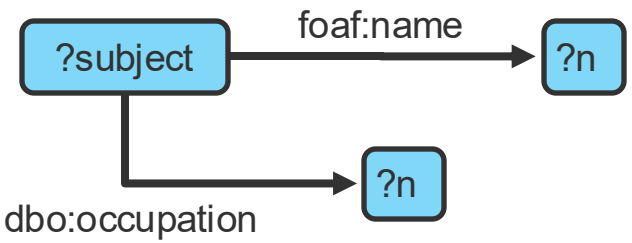

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject WHERE {
      ?subject foaf:name ?name .
} LIMIT 10 OFFSET 5
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Finds all subjects that have a name (foaf:name) or that have an occuation.




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dbo: <
http://dbpedia.org/ontology/> >

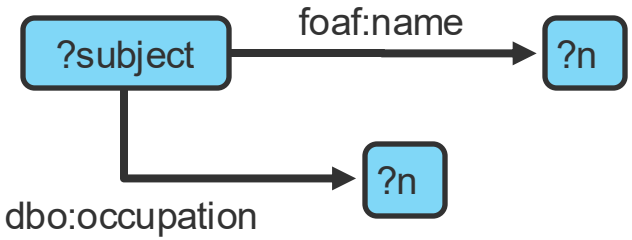
SELECT DISTINCT ?subject WHERE {
      ?subject foaf:name ?name .
 OPTIONAL { ?subject dbo:occupation
?trabajo . }

}
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Find all subjects that have a name (foaf:name) or have an occupation or are called "Leonardo da Vinci".




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dbo: < http://dbpedia.org/ontology/> >

SELECT DISTINCT ?subject WHERE {
      ?subject foaf:name ?name .
 OPTIONAL {
      ?subject dbo:occupation ?trabajo .
}
 OPTIONAL {
      ?subject foaf:name “Leonardo da Vinci” .
}
}
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Find all subjects with the name "Leo" or "Leonardo da Vinci".




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject WHERE {
      ?subject foaf:name ?name .
      VALUES ?name { "Leo" "Leonardo da Vinci" }
}
```

SPARQL FUNCTIONS

ASK, **SELECT**, DESCRIBE, CONSTRUCT



RDF-terms based



Numeric-based



Date-based



String-based



Hash-based

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Find all subjects with a name (foaf:name) beginning with "Leo".




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject WHERE {
        ?subject foaf:name ?name .
        FILTER (strstarts(str(?name), “Leo”))
}
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Find all subjects that have a name (foaf:name) and sort the results by that name

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject WHERE {
      ?subject foaf:name ?name .
} ORDER BY ?name
```



SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Find and return the number of names each subject has.




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject (count(distinct ?name) as ?count) WHERE {

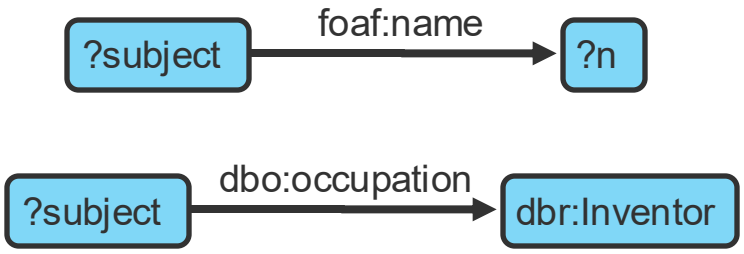
        ?subject foaf:name ?name .

} GROUP BY ?subject
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Find and return the subjects that have a name and join them with those that have as an occupation inventors




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject WHERE {
 {
      ?subject foaf:name ?name .
  } UNION {
      ?subject dbo:occupation dbr:Inventor .
  }
}
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Find triples within a named graph



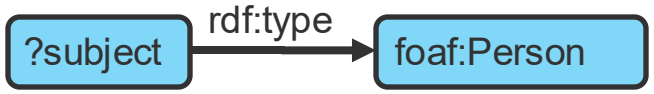

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?graph ?subject WHERE {
 GRAPH ?graph {
      ?subject foaf:name ?n.
  }
}
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

- Find triples in a remote triple store




```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?subject WHERE {
   SERVICE <https://dbpedia.org/sparql> {
        ?subject a foaf:Person .
   }
}
```

SPARQL

ASK, **SELECT**, DESCRIBE, CONSTRUCT

CSV

```
[variables]
[values]
```

XML

```
…
```

JSON

```
{
  "head": { "vars": [ [variables] ] },
  "results": {
    "bindings": [
      {
        ”variable": {
          "type": "uri/literal",
          "value": ”valor_variable"
        }
      }
    ]}}
```

SPARQL

ASK, SELECT, **DESCRIBE**, CONSTRUCT

```
DESCRIBE <URI>
```

```
[PREFIXES]

DESCRIBE [VARIABLES] WHERE {
        [Graph Patters]
        [FILTER]
}
```

- Find all triples of the URI dbr:Leonardo_da_Vinci

```
DESCRIBE <dbr:Leonardo_da_Vinci>
```

SPARQL

ASK, SELECT, DESCRIBE, **CONSTRUCT**

```
[PREFIXES]

CONSTRUCT { [VARIABLES] }
WHERE {
      [Graph Patters]
}
```

- Construct an RDF fragment where Leonardo da Vinci is assigned an additional name "Leo"

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
CONSTRUCT {
      ?subject ?predicate ?object .
      ?subject foaf:name “Leo” .
} WHERE {
      ?subject foaf:name ”Leonardo da Vinci” .
       ?subject ?predicate ?object .
}
```

SPARQL

Protocol

- The protocol specifies how to send queries over the Web.

	HTTP Method	Query String Parameters	Request Content Type	Request Message Body
query via GET	GET	`query` (exactly 1) `default-graph-uri` (0 or more) `named-graph-uri` (0 or more)	None	None
query via URL-encoded POST	POST	None	`application/x-www-form-urlencoded`	URL-encoded, ampersand-separated query parameters. `query` (exactly 1) `default-graph-uri` (0 or more) `named-graph-uri` (0 or more)
query via POST directly	POST	`default-graph-uri` (0 or more) `named-graph-uri` (0 or more)	`application/sparql-query`	Unencoded SPARQL query string

- Typically, triple-based databases usually publish a SPARQL endpoint
  - A URL capable of resolving queries

SPARQL

Well-known endpoints

- DBPEDIA (https://es.dbpedia.org/sparql)
- AEMET (http://aemet.linkeddata.es/sparql)
- Spanish government (https://datos.gob.es/es/sparql)
- …

- SPARQL cheat sheet
  - https://www.iro.umontreal.ca/~lapalme/ift6281/sparql-1_1-cheat-sheet.pdf